MLP Assignment

Name: Ella Fahey

Student Number: C22396101

```python
class XOR_MLP:
    def __init__(self, input_size=2, hidden_size=2, output_size=1):
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.output_size = output_size

        # XOR-specific inputs and outputs
        self.train_inputs = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
        self.train_outputs = np.array([[0], [1], [1], [0]])

        np.random.seed(23)

        # Initialize weights and biases
        self.w2 = np.random.randn(hidden_size, input_size)
        self.b2 = np.random.randn(hidden_size, 1)

        self.w3 = np.random.randn(output_size, hidden_size)
        self.b3 = np.random.randn(output_size, 1)

    def feedforward(self, xs):
        """Feedforward computation."""
        a2s = sigm(self.w2 @ xs + self.b2)
        a3s = sigm(self.w3 @ a2s + self.b3)
        return a3s

    def backprop(self, xs, ys):
        """Backpropagation for gradient calculation."""
        del_w2 = np.zeros(self.w2.shape, dtype=float)
```

jupyter XOR_MLP v1 (1) Last Checkpoint: 31 minutes ago

File Edit View Run Kernel Settings Help

Code JupyterLab Python 3 (ipykern

```python
        """Backpropagation for gradient calculation."""
        del_w2 = np.zeros(self.w2.shape, dtype=float)
        del_b2 = np.zeros(self.b2.shape, dtype=float)

        del_w3 = np.zeros(self.w3.shape, dtype=float)
        del_b3 = np.zeros(self.b3.shape, dtype=float)

        cost = 0.0
        for x, y in zip(xs, ys):
            a1 = x.reshape(self.input_size, 1)
            z2 = self.w2 @ a1 + self.b2
            a2 = sigm(z2)

            z3 = self.w3 @ a2 + self.b3
            a3 = sigm(z3)

            delta3 = (a3 - y) * sigm_deriv(z3)
            delta2 = sigm_deriv(z2) * (self.w3.T @ delta3)

            del_b3 += delta3
            del_w3 += delta3 @ a2.T

            del_b2 += delta2
            del_w2 += delta2 @ a1.T

            cost += ((a3 - y) ** 2).sum()

        n = len(ys)
        return del_b2 / n, del_w2 / n, del_b3 / n, del_w3 / n, cost / n
```

File Edit View Run Kernel Settings Help

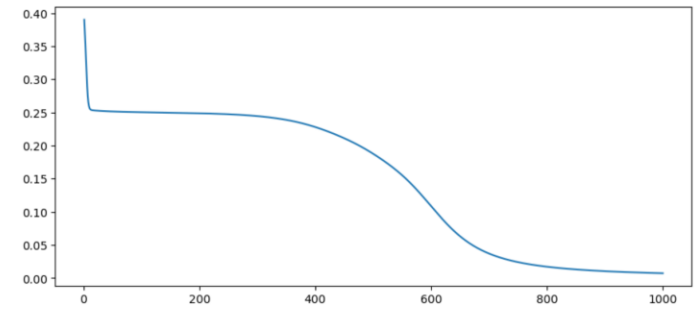
Code JupyterLab Python 3 (ipykernel)

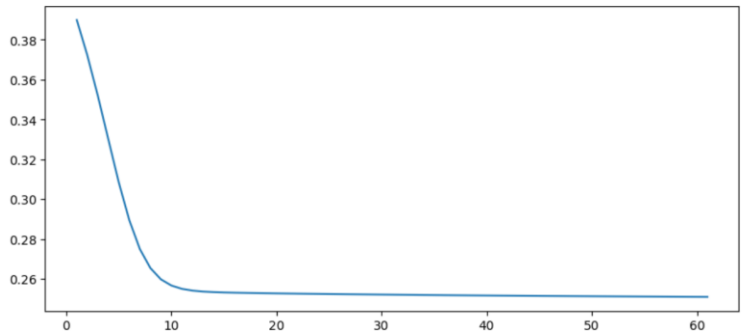
```python
    def train(self, epochs, eta):
        """Training using gradient descent."""
        xs = self.train_inputs
        ys = self.train_outputs
        cost = np.zeros(epochs)

        for e in range(epochs):
            d_b2, d_w2, d_b3, d_w3, cost[e] = self.backprop(xs, ys)

            self.b2 -= eta * d_b2
            self.w2 -= eta * d_w2
            self.b3 -= eta * d_b3
            self.w3 -= eta * d_w3

        return cost
```

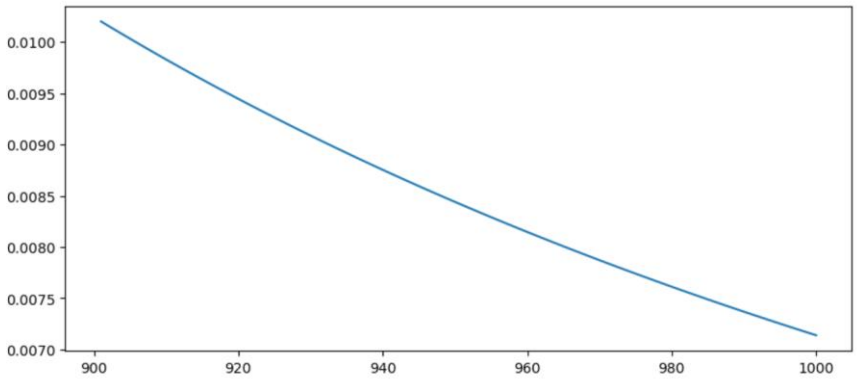
[29]:

```python
xor = XOR_MLP()
xs = xor.train_inputs.T

print(xor.feedforward(xs))

epochs = 1000
c = xor.train(epochs, 3.0)

print(xor.feedforward(xs))

x_axis = np.linspace(1, epochs, epochs, dtype=int)
fig, axs = plt.subplots(3,1,figsize=(10,15))
```

```
[[0.13441229 0.10816814 0.14522425 0.12453942]]
[[0.08467026 0.91859922 0.91853706 0.08963025]]
[<matplotlib.lines.Line2D at 0x15e2e97c680>]
```







Explanation:

As demonstrated above, thsi implements a Multi-Layer Perceptron (MLP) designed to solve the XOR problem. This code includes an input layer with two neurons (for binary inputs), a hidden layer with two neurons (to facilitate non-linear transformations), and an output layer with one neuron (to predict the XOR output). The class XOR_MLP initialises the training data, representing the truth table for XOR, as well as random weights and biases for the connections between layers. The initial weights are set using a fixed seed to ensure consistent results across runs.

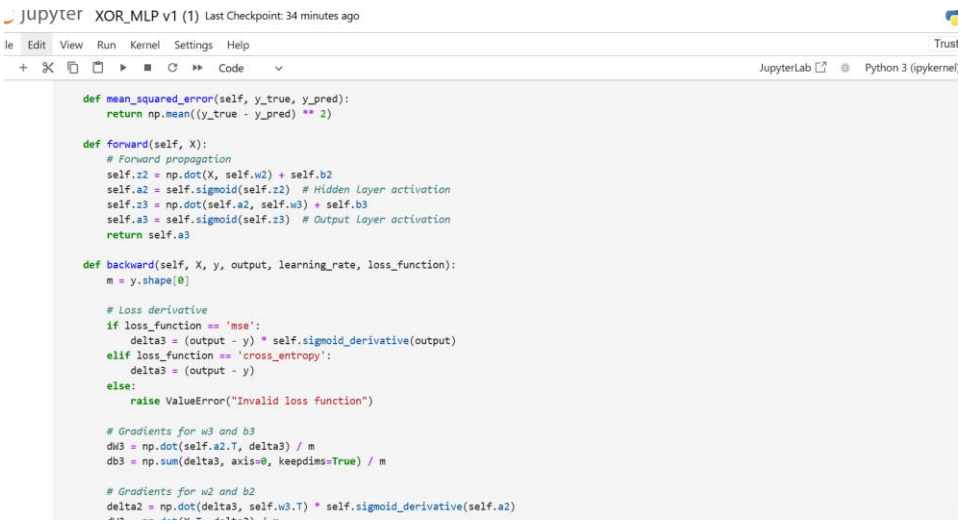
The feedforward pass computes activations layer by layer, applying the sigmoid function to introduce non-linearity. Inputs are transformed through the hidden layer, and the resultant activations are passed to the output layer to produce the final predictions. Prior to training, the outputs are essentially random due to the randomised initial weights. The backpropagation method computes gradients for weights and biases by propagating the error from the output layer back through the hidden layer. It calculates adjustments to minimise the error, quantified by the mean squared cost function, over the entire dataset. These gradients are used in the train

method, where gradient descent is applied to iteratively adjust weights and biases over a specified number of epochs. The cost function is monitored across epochs, showing a steady decline as the network learns.

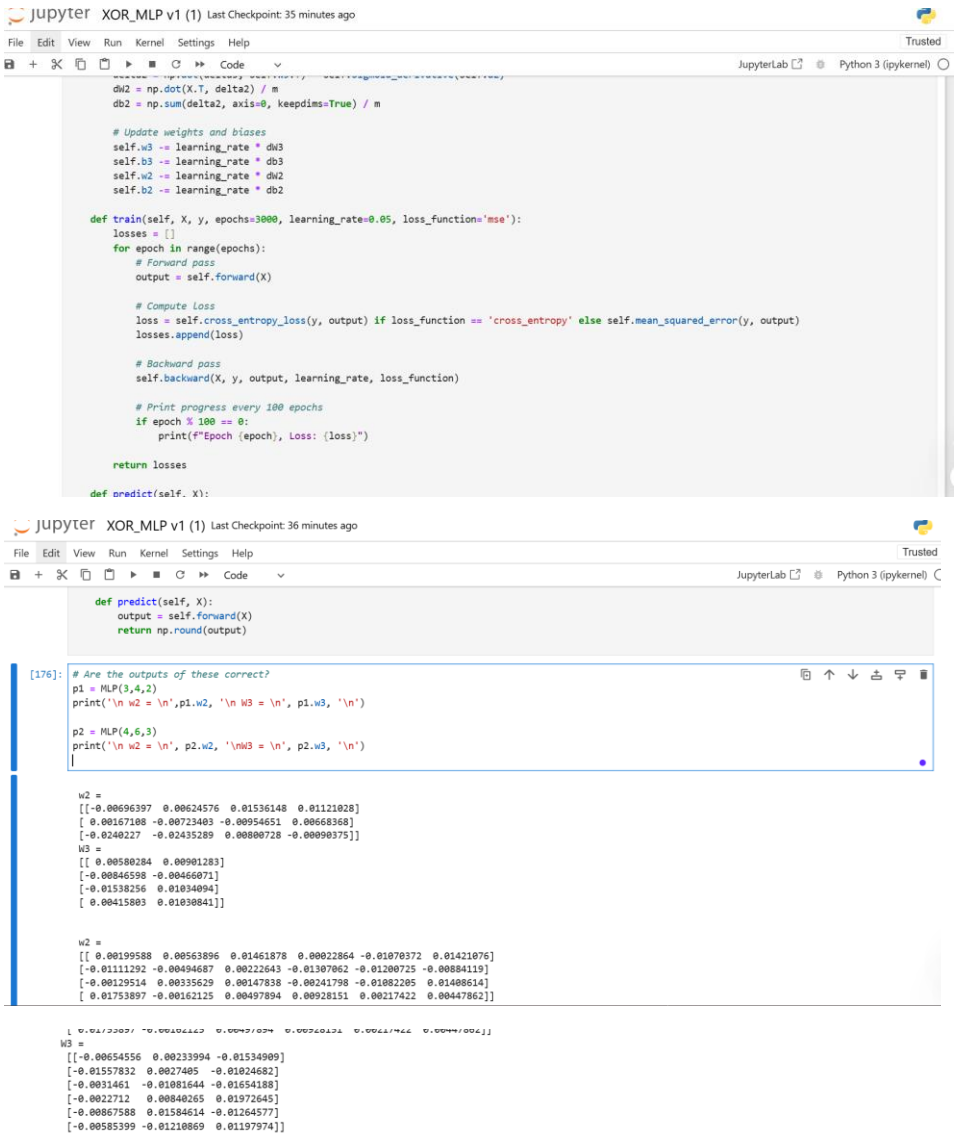
Before training, the outputs of the network fail to represent the XOR function, as the weights have not yet been optimised. However, after training with a sufficiently high learning rate and adequate epochs, the network converges, producing outputs that closely approximate the desired XOR values of [0, 1, 1, 0]. The cost curve, plotted during training, reveals the learning dynamics, with a rapid initial decline followed by gradual stabilisation as the model approaches its optimal state. This solution highlights the critical role of the hidden layer in mapping inputs to a higher-dimensional space, enabling the separation of XOR classes. It also demonstrates the importance of backpropagation and gradient descent in minimising the error and achieving effective learning. By training this simple neural network, the code successfully models the XOR operation, showcasing the capability of an MLP to solve non-linear problems.

Exercise 1:

```
# A more general purpose MLP with m input neurons, n hidden neurons and o output neurons
# You must complete this code yourself

import numpy as np
import pandas as pd

class MLP:
    def __init__(self, m, n, o):
        self.m = m  # Number of input neurons
        self.n = n  # Number of hidden neurons
        self.o = o  # Number of output neurons

        # Initialize weights and biases for the layers
        self.w2 = np.random.randn(self.m, self.n) * 0.01  # Weights between input and hidden layer
        self.b2 = np.zeros((1, self.n))  # Biases for hidden layer
        self.w3 = np.random.randn(self.n, self.o) * 0.01  # Weights between hidden and output layer
        self.b3 = np.zeros((1, self.o))  # Biases for output layer

    def sigmoid(self, z):
        return 1 / (1 + np.exp(-z))

    def sigmoid_derivative(self, z):
        return z * (1 - z)

    def cross_entropy_loss(self, y_true, y_pred):
        m = y_true.shape[0]
        epsilon = 1e-12
        y_pred = np.clip(y_pred, epsilon, 1 - epsilon)
        return -np.sum(y_true * np.log(y_pred) + (1 - y_true) * np.log(1 - y_pred)) / m
```




```
    def mean_squared_error(self, y_true, y_pred):
        return np.mean((y_true - y_pred) ** 2)

    def forward(self, X):
        # Forward propagation
        self.z2 = np.dot(X, self.w2) + self.b2
        self.a2 = self.sigmoid(self.z2)  # Hidden layer activation
        self.z3 = np.dot(self.a2, self.w3) + self.b3
        self.a3 = self.sigmoid(self.z3)  # Output layer activation
        return self.a3

    def backward(self, X, y, output, learning_rate, loss_function):
        m = y.shape[0]

        # Loss derivative
        if loss_function == 'mse':
            delta3 = (output - y) * self.sigmoid_derivative(output)
        elif loss_function == 'cross_entropy':
            delta3 = (output - y)
        else:
            raise ValueError("Invalid loss function")

        # Gradients for w3 and b3
        dW3 = np.dot(self.a2.T, delta3) / m
        db3 = np.sum(delta3, axis=0, keepdims=True) / m

        # Gradients for w2 and b2
        delta2 = np.dot(delta3, self.w3.T) * self.sigmoid_derivative(self.a2)
```



Explanation:

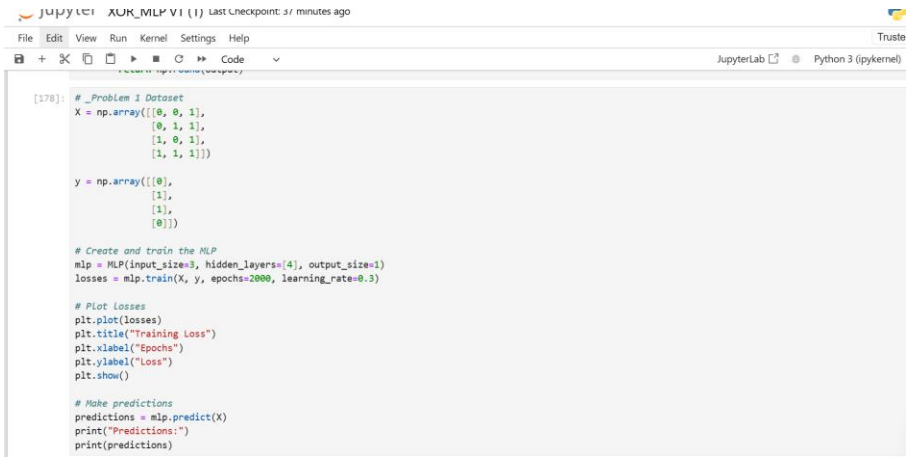
Here the program presents a general-purpose implementation of MLP, allowing flexibility in the number of input, hidden, and output neurons. The MLP is initialised with randomly assigned weights and biases, using a small random initialisation to facilitate efficient training. The code includes methods for forward propagation, loss calculation, backpropagation, and training, offering support for two loss functions: mean squared error (MSE) and cross-entropy loss. The forward method calculates activations at each layer using the sigmoid function, a standard choice for introducing non-linearity, while the backpropagation method computes gradients for the weights and biases to adjust them during training.

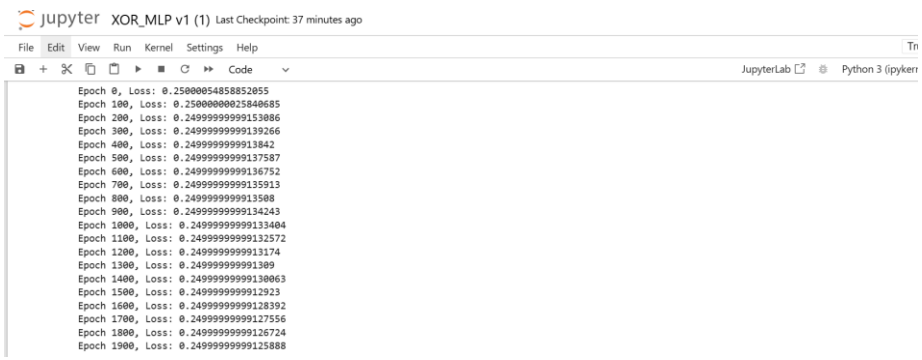
The loss function serves as a metric for the network's performance. For binary classification tasks, cross-entropy is preferred, as it better handles probabilities compared to MSE. The training loop iteratively refines the weights and biases over a specified number of epochs using gradient descent, reducing the error and improving the network's predictions. The train method tracks and reports the loss at regular intervals, giving insight into the convergence process.

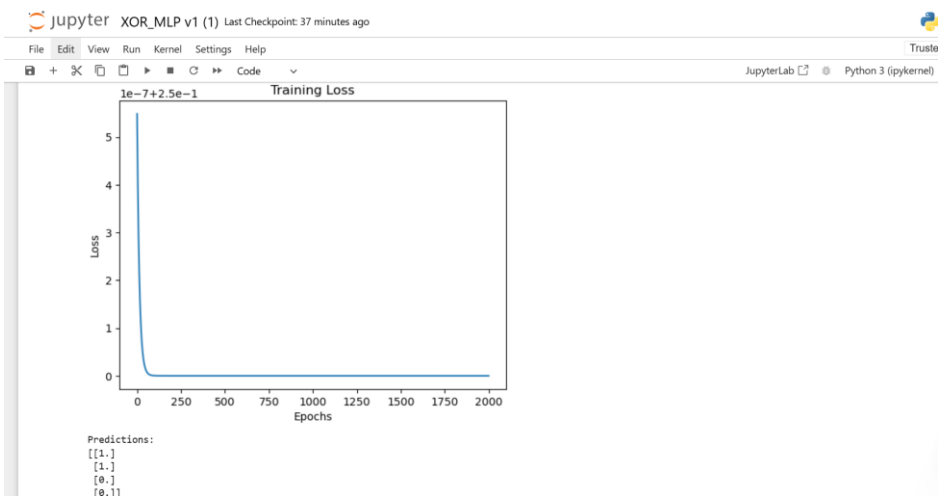
Finally, the predict function provides a rounded output, translating the network's continuous outputs into discrete classes.

Testing the MLP with two configurations of neurons reveals the initial weights, which are small and random. The program is designed to learn complex mappings between inputs and outputs through training. The modularity of this implementation makes it interchangeable for various use cases, from simple binary classifications to multi-class problems, depending on the specified architecture and training parameters. By combining key elements like backpropagation and gradient descent with modular design, the code highlights the fundamental principles of MLPs and their adaptability for diverse machine learning tasks.

Problem 1: epochs 2000, learning rate 0.3







Explanation:

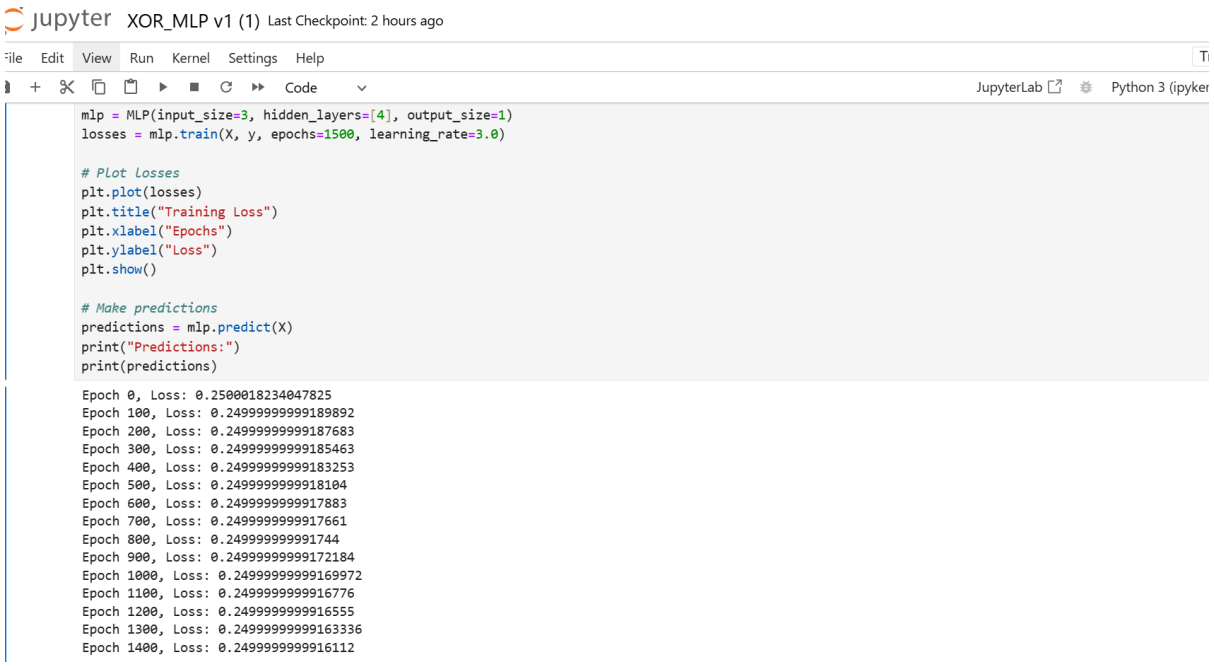
This allows for a parameterised architecture with a flexible number of input, hidden, and output neurons. The design supports one or more hidden layers, each defined by the user during instantiation. Weights and biases are initialised for each layer, using small random values for weights and zero values for biases. Forward propagation is carried out by applying the sigmoid activation function iteratively across layers, with activations stored at each step for later use during backpropagation.

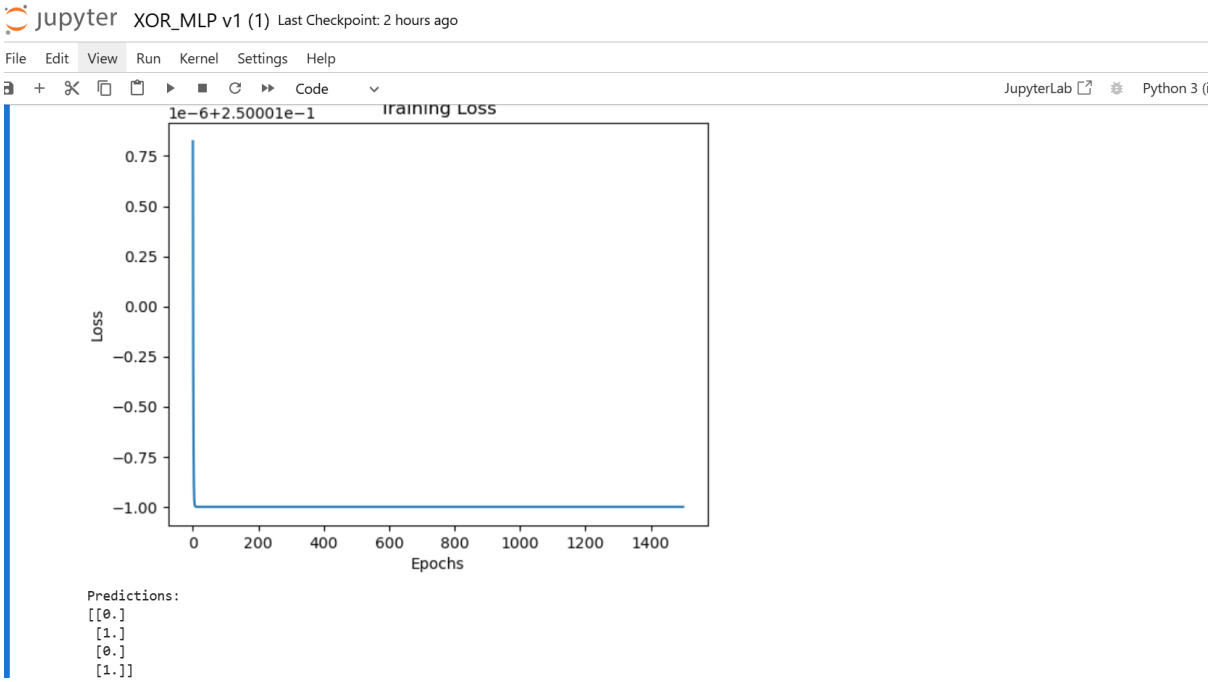
The training process involves minimising the loss function—either mean squared error (MSE) or cross-entropy—through gradient descent. During backpropagation, the gradients of the loss with respect to weights and biases are computed layer-by-layer, starting from the output and propagating back to the input. Updates are then applied to reduce the loss iteratively over a user-defined number of epochs.

To demonstrate the model, a dataset with binary inputs and outputs is used. A single hidden layer of four neurons is specified, and the model is trained for 2000 epochs using a learning rate of 0.3. The loss is recorded at each epoch, and its plot reveals the network's convergence over time. Following training, the network is tested on the same input dataset, producing predictions that match the expected outputs.

This implementation underscores the versatility of MLPs for tasks like binary classification. The parameterisation of hidden layers allows the model to adapt to different complexities, while its modular structure facilitates extensions to more complex datasets and activation functions. Moreover, visualising the loss curve helps assess the training process and detect potential issues, such as slow convergence or overfitting.

Problem 1 with 1500 Epochs and a learning rate of 3.0:





Explanation:

In the altered code, the goal is to train a MLP model to classify data based on the input features, with training performed for 1500 epochs and a learning rate of 3.0. The key changes from the original setup, which had 2000 epochs and a learning rate of 0.3, include reducing the number of epochs and significantly increasing the learning rate. With fewer epochs (1500), the model has fewer opportunities to adjust its weights and biases, which may result in the model not fully learning the underlying patterns in the data. Consequently, the model does not converge as well as it did with 2000 epochs, leading to higher loss values and less accurate predictions. On the other hand, the higher learning rate (3.0) causes the model to make larger updates to the weights during training. While this can speed up convergence, it also increases the risk of overshooting the optimal solution, potentially leading to instability and fluctuations in the training process.

As a result, the model produces predictions of [0, 1, 0, 1]. This suggests that the model hasn't fully learned the data, likely due to the combination of fewer epochs and a higher learning rate. In summary, the reduced number of epochs and increased learning rate resulted in less accurate predictions and slower or convergence, highlighting the trade-off between training time and model accuracy.

Problem 2: Neural network with 3 inputs and 2 outputs

jupyter XOR_MLP v1 (1) Last Checkpoint: 38 minutes ago

File Edit View Run Kernel Settings Help

Code JupyterLab Python 3 (ipykern

[85]:
```
# Problem 2: Neural Network with 3 inputs and 2 outputs
def problem_2_training():
    # Define dataset for Problem 2
    X = np.array([[1, 1, 0],
                  [1, -1, -1],
                  [-1, 1, 1],
                  [-1, -1, 1],
                  [0, 1, -1],
                  [0, -1, -1],
                  [1, 1, 1]])
    y = np.array([[1, 0],
                  [0, 1],
                  [1, 1],
                  [1, 0],
                  [1, 0],
                  [1, 1],
                  [1, 1]])

    # Create and train the MLP
    mlp_problem_2 = MLP(input_size=3, hidden_layers=[6], output_size=2)  # Use a list for hidden_layers
    problem_2_losses = mlp_problem_2.train(X, y, epochs=3000, learning_rate=0.05, loss_function='cross_entropy')

    # Plot loss
    plt.plot(problem_2_losses)
    plt.title('Loss Curve for Problem 2')
    plt.xlabel('Epochs')
    plt.ylabel('Loss')
    plt.show()
```

jupyter XOR_MLP v1 (1) Last Checkpoint: 38 minutes ago

File Edit View Run Kernel Settings Help

Code JupyterLab Python 3 (ipykernel)

```
    # Predict example
    example = np.array([[1, 0, 1]])  # Example data
    prediction = mlp_problem_2.predict(example)
    print("Prediction for example:", prediction)

# Call Problem 2 training
problem_2_training()
```

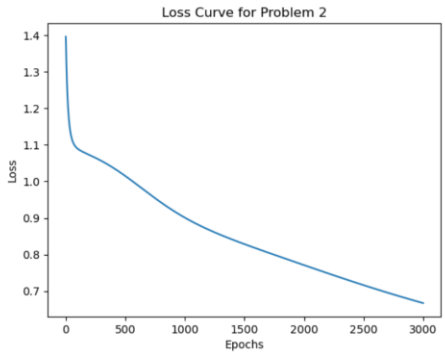
```
Epoch 0, Loss: 1.3962781531143413
Epoch 100, Loss: 1.0893456804125372
Epoch 200, Loss: 1.0721351148711455
Epoch 300, Loss: 1.0559157401847 92
Epoch 400, Loss: 1.036770797427691
Epoch 500, Loss: 1.0146687891983792
Epoch 600, Loss: 0.9905440978129649
Epoch 700, Loss: 0.9658745546590068
Epoch 800, Loss: 0.9421079705947077
Epoch 900, Loss: 0.920217746996979
Epoch 1000, Loss: 0.9005890421341493
Epoch 1100, Loss: 0.8831533656902166
Epoch 1200, Loss: 0.8675922426916107
Epoch 1300, Loss: 0.8534998376760873
Epoch 1400, Loss: 0.8404798925636577
Epoch 1500, Loss: 0.8281920014981894
Epoch 1600, Loss: 0.8163679379575662
Epoch 1700, Loss: 0.8048127027834947
Epoch 1800, Loss: 0.7933983624640873
Epoch 1900, Loss: 0.7820544901831467
Epoch 2000, Loss: 0.7707569208184976
```

jupyter XOR_MLP v1 (1) Last Checkpoint: 38 minutes ago

File Edit View Run Kernel Settings Help

Code JupyterLab Python 3 (ipykern

```
Epoch 2800, Loss: 0.6856095941694306
Epoch 2900, Loss: 0.6760616830342491
```




```
Prediction for example: [[1. 1.]]
```

Explanation:

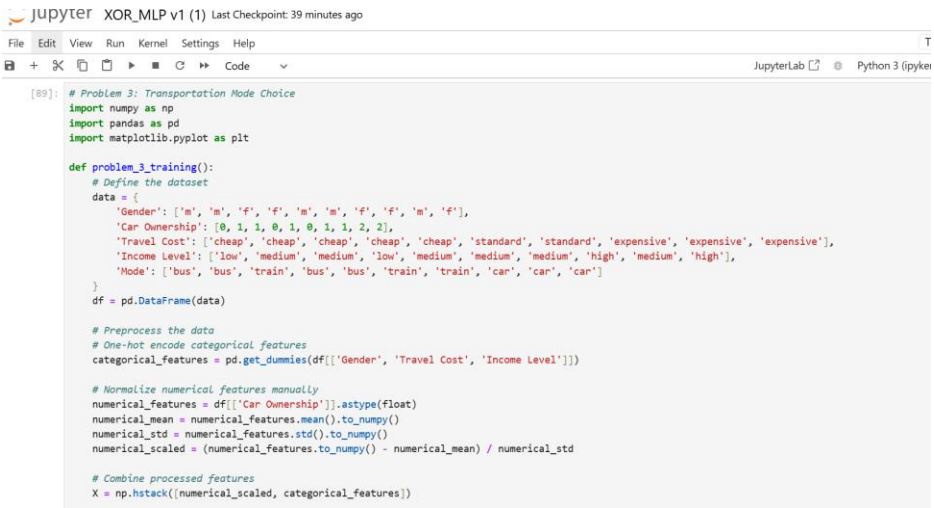
This code once again makes use of MLP. In this case, to solve a problem involving a dataset with three input features and two output labels. The dataset includes seven training samples, each consisting of three input values and corresponding two-dimensional target outputs. The neural network is configured with three input neurons, one hidden layer containing six neurons, and two output neurons. The use of a single hidden layer provides the network with sufficient capacity to model the relationships between the inputs and outputs.

The training process uses the cross-entropy loss function, which is particularly suitable for multi-output classification tasks. During training, forward propagation computes the activations for each layer, and backpropagation adjusts the weights and biases to minimise the loss. Training is carried out with 3000 epochs with a learning rate of 0.05, iteratively reducing the loss function and improving the model's predictions.

A plot of the loss curve is displayed, illustrating the decline in loss over epochs. This provides a visual indication of the network's convergence. Although there is an initial large drop in loss, the curve demonstrates a smooth and steady reduction in loss, suggesting effective learning. Sharp oscillations might indicate issues like an unsuitable learning rate or insufficient training.

Finally, the network is tested with an example input [[1, 0, 1]]. The prediction output is printed, showing a two-dimensional array representing the network's confidence in each of the two output classes. The model has learned to associate certain patterns in the input features with the output labels during training. The network's forward propagation computes activations through the layers. The output layer's activation values, after applying the sigmoid function, yield values close to 1 for both output neurons, which are rounded to [1, 1]. This result suggests that the network has identified the input as strongly related to both output classes.

Problem 3: Transportation Mode Choice



jupyter XOR_MLP v1 (1) Last Checkpoint: 39 minutes ago

File Edit View Run Kernel Settings Help

Code

JupyterLab Python 3 (ipykernel)

```
# One-hot encode the target variable
target_mapping = {mode: idx for idx, mode in enumerate(df['Mode'].unique())}
y_encoded = df['Mode'].map(target_mapping).values
y = np.eye(len(target_mapping))[y_encoded]

# Train the MLP
mlp_problem_3 = MLP(input_size=X.shape[1], hidden_layers=[8], output_size=y.shape[1])
problem_3_losses = mlp_problem_3.train(X, y, epochs=3000, learning_rate=0.05, loss_function='cross_entropy')

# Plot the loss curve
plt.plot(problem_3_losses)
plt.title('Loss Curve for Problem 3')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.show()

# Predict for a new instance
example_data = {
    'Gender': 'f',  # Female
    'Car Ownership': 0,
    'Travel Cost': 'expensive',
    'Income Level': 'medium'
}

# Manually encode the example
example_categorical = pd.get_dummies(pd.DataFrame([example_data])).reindex(columns=categorical_features.columns, fill_value=0).values.astype(np.float64)
example_numerical = (np.array([[example_data['Car Ownership']]], dtype=np.float64) - numerical_mean) / numerical_std
example = np.hstack([example_numerical, example_categorical])
```

jupyter XOR_MLP v1 (1) Last Checkpoint: 40 minutes ago

File Edit View Run Kernel Settings Help

Code

JupyterLab Python 3 (ipykernel)

```
    prediction = mlp_problem_3.predict(example)
    mode_prediction = {v: k for k, v in target_mapping.items()}[np.argmax(prediction)]
    print("Prediction for the instance (Female):", mode_prediction)

# Call Problem 3 training
problem_3_training()
```

```
Epoch 0, Loss: 2.079753165029378
Epoch 100, Loss: 1.8932477003999
Epoch 200, Loss: 1.8889577959336106
Epoch 300, Loss: 1.8789156446987931
Epoch 400, Loss: 1.8524054556467284
Epoch 500, Loss: 1.7881953239266148
Epoch 600, Loss: 1.667031827888381
Epoch 700, Loss: 1.5027639141910263
Epoch 800, Loss: 1.3414342399420618
Epoch 900, Loss: 1.2145754634401278
Epoch 1000, Loss: 1.1175602852815 68
Epoch 1100, Loss: 1.0329387577754463
Epoch 1200, Loss: 0.9467660090698639
Epoch 1300, Loss: 0.8535614236145681
Epoch 1400, Loss: 0.7560708628152482
Epoch 1500, Loss: 0.6602023921025367
Epoch 1600, Loss: 0.5710232474030726
Epoch 1700, Loss: 0.49163026695863865
Epoch 1800, Loss: 0.42316025221316905
Epoch 1900, Loss: 0.36533084769096585
Epoch 2000, Loss: 0.3170791411993815
```

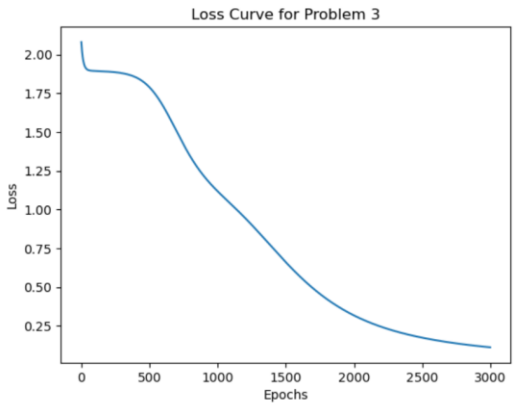
jupyter XOR_MLP v1 (1) Last Checkpoint: 40 minutes ago

File Edit View Run Kernel Settings Help

Code

JupyterLab

```
Epoch 2800, Loss: 0.13120642113517986
Epoch 2900, Loss: 0.12076955723422374
```




```
Prediction for the instance (Female): train
```

Delimiter: ,

	Gender	Car Ownership	Travel Cost	Income Level	Mode
1	m	0	cheap	low	bus
2	m	1	cheap	medium	bus
3	f	1	cheap	medium	train
4	f	0	cheap	low	bus
5	m	1	cheap	medium	bus
6	m	0	standard	medium	train
7	f	1	standard	medium	train
8	f	1	expensive	high	car
9	m	2	expensive	medium	car
10	f	2	expensive	high	car

The output prediction for the transportation mode in this code is determined by the model's ability to learn patterns in the dataset and map input features to the target classes. In this program, the dataset encodes information about individuals' gender, car ownership, travel costs, and income levels, with the goal of predicting their preferred mode of transport: bus, train, or car.

The data undergoes preprocessing, where categorical variables (like gender, travel cost, and income level) are transformed into numerical format using one-hot encoding. This ensures that the model can interpret these non-numeric inputs effectively. Numerical features, such as car ownership, are normalised to ensure that their scales do not dominate the learning process. These processed features are combined into a single input array, X, while the transportation modes are encoded into a one-hot matrix, y, to represent the target classes.

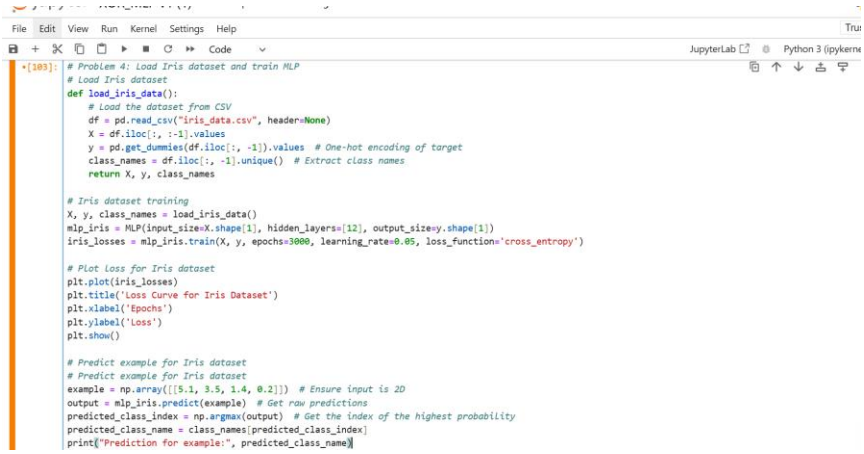
The MLP model is then trained using these features. During training, the model adjusts its weights to minimise a loss function (cross-entropy loss), which measures the disparity between the predicted and actual modes of transport. As training progresses, the loss decreases, indicating the model is learning patterns that align input features with their corresponding transport modes.

For prediction, a new input instance is provided, which represents an individual with the characteristics of female, no car ownership, expensive travel costs, and possessing a medium income level. This input is processed similarly to the training data and fed into the model. The model outputs probabilities for each mode of transport, and the mode with the highest probability is selected as the prediction.

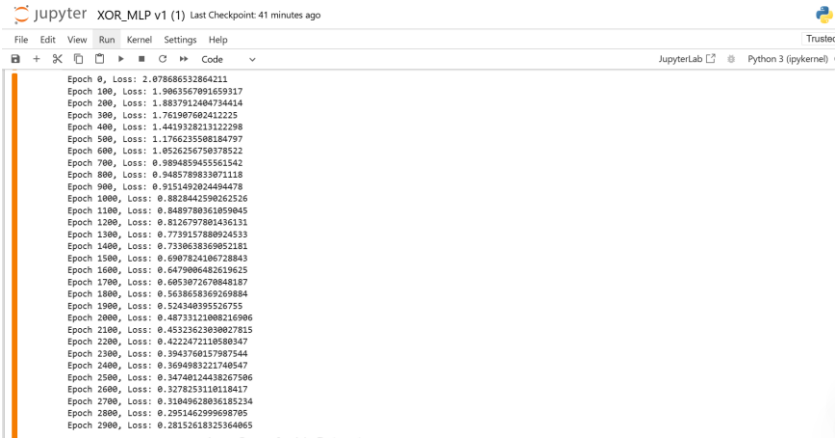
In this case, the prediction reflects the model's understanding of the given input and its alignment with patterns in the dataset. The output probabilities correspond to the likelihood of each transportation mode given the input data. For this specific input, the highest probability is assigned to "train," indicating that the model has learned to associate this combination of features—female gender, zero car ownership, expensive travel costs, and medium income level—with a preference for train travel. The final output not only showcases the model's capability to generalise but also highlights how it interprets nuanced relationships between features and outcomes.

Problem 4: Iris Data Set

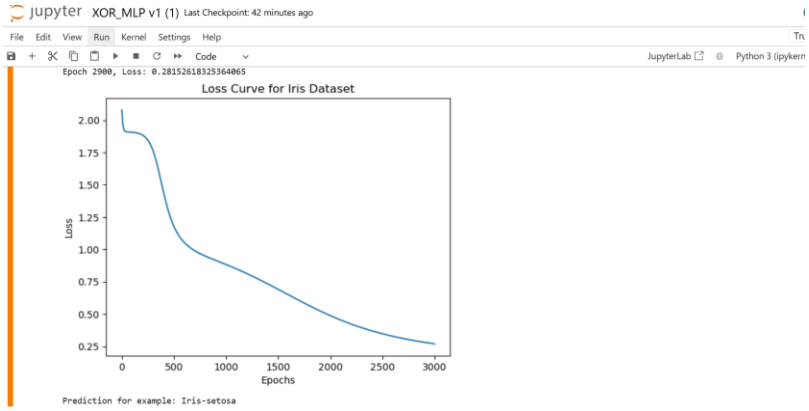
12 Hidden layers, learning rate 0.05



```
# Problem 4: Load Iris dataset and train MLP
# Load Iris dataset
def load_iris_data():
    # Load the dataset from CSV
    df = pd.read_csv("iris_data.csv", header=None)
    X = df.iloc[:, :-1].values
    y = pd.get_dummies(df.iloc[:, -1]).values  # One-hot encoding of target
    class_names = df.iloc[:, -1].unique()  # Extract class names
    return X, y, class_names

# Iris dataset training
X, y, class_names = load_iris_data()
mlp_iris = MLP(input_size=X.shape[1], hidden_layers=[12], output_size=y.shape[1])
iris_losses = mlp_iris.train(X, y, epochs=3000, learning_rate=0.05, loss_function='cross_entropy')

# Plot Loss for Iris dataset
plt.plot(iris_losses)
plt.title('Loss Curve for Iris Dataset')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.show()

# Predict example for Iris dataset
# Predict example for Iris dataset
example = np.array([[5.1, 3.5, 1.4, 0.2]])  # Ensure input is 2D
output = mlp_iris.predict(example)  # Get raw predictions
predicted_class_index = np.argmax(output)  # Get the index of the highest probability
predicted_class_name = class_names[predicted_class_index]
print("Prediction for example:", predicted_class_name)
```



```
Epoch 0, Loss: 2.07868653864211
Epoch 100, Loss: 1.9063567091659317
Epoch 200, Loss: 1.8837912404734414
Epoch 300, Loss: 1.761907602412225
Epoch 400, Loss: 1.4419328213122298
Epoch 500, Loss: 1.1766235508184797
Epoch 600, Loss: 1.0526256750378522
Epoch 700, Loss: 0.9894859455561542
Epoch 800, Loss: 0.9485789833071118
Epoch 900, Loss: 0.9151492024494478
Epoch 1000, Loss: 0.8828442590262526
Epoch 1100, Loss: 0.8489780361059045
Epoch 1200, Loss: 0.8126797801436131
Epoch 1300, Loss: 0.7739157880924533
Epoch 1400, Loss: 0.7330638369052181
Epoch 1500, Loss: 0.6907824106728843
Epoch 1600, Loss: 0.6479006482619625
Epoch 1700, Loss: 0.6053072670848187
Epoch 1800, Loss: 0.5638658369269884
Epoch 1900, Loss: 0.5243403955267555
Epoch 2000, Loss: 0.4873312100821690 6
Epoch 2100, Loss: 0.4532362303002781 5
Epoch 2200, Loss: 0.4222472110580347
Epoch 2300, Loss: 0.3943760157987544
Epoch 2400, Loss: 0.3694983221740547
Epoch 2500, Loss: 0.3474012443826750 6
Epoch 2600, Loss: 0.3278253110118417
Epoch 2700, Loss: 0.3104962803618523 4
Epoch 2800, Loss: 0.2951462999698705
Epoch 2900, Loss: 0.2815261832536406 5
```

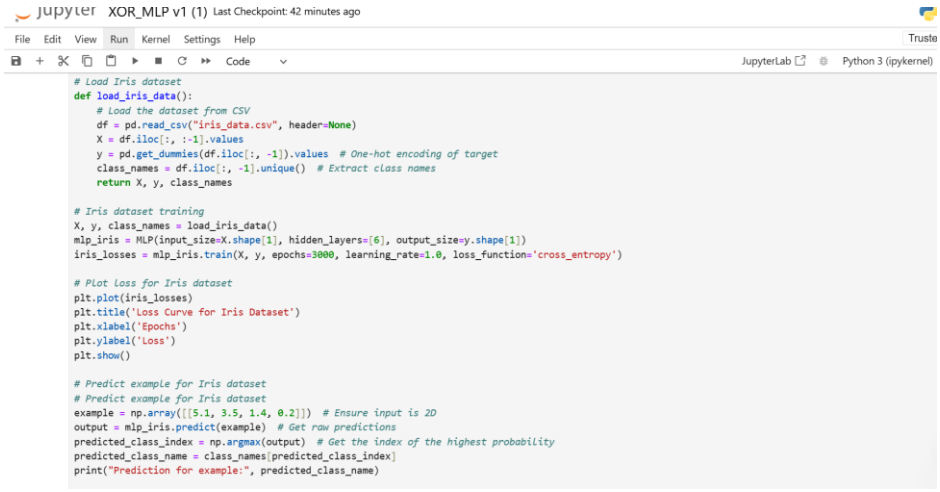


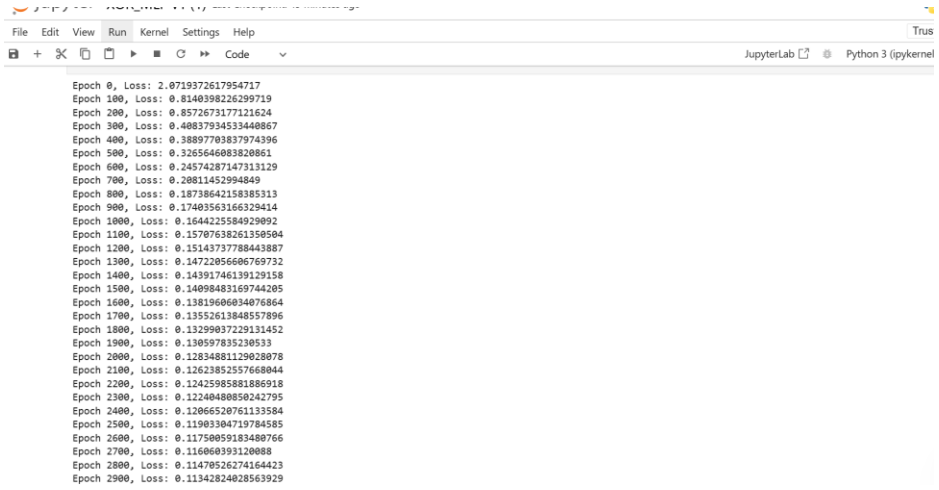

```
Prediction for example: Iris-setosa
```

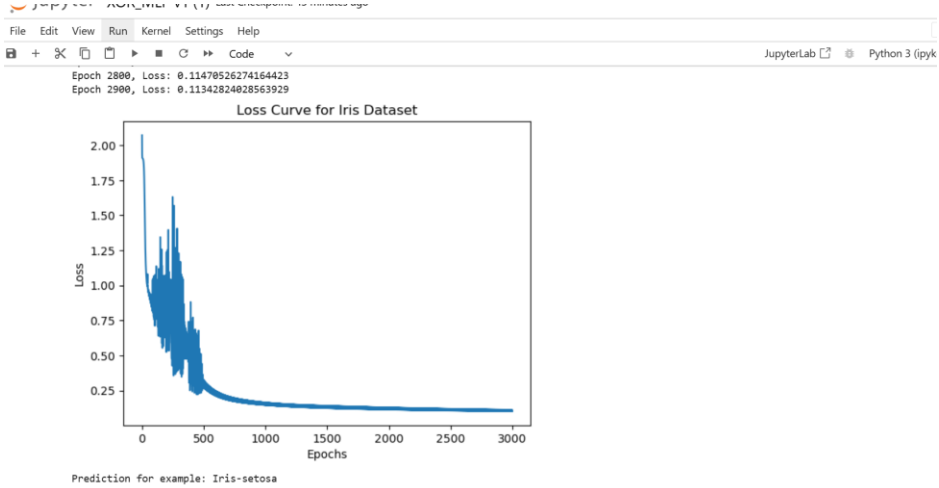
This code loads the Iris dataset, trains a Multi-Layer Perceptron model to classify flower species based on features like sepal and petal length and width, and then visualizes the model's performance during training. First, the dataset is loaded from the iris_data.csv file where the features are separated from the target variable (species of the flower). The target variable is one-hot encoded, and the dataset is split into feature vectors (X) and labels (y). The MLP model is initialized with 4 input neurons (for the 4 features), 12 neurons in a hidden layer, and 3 output neurons (one for each flower species). The model is trained for 3000 epochs using the cross-entropy loss function. During training, the model's performance is tracked, and the loss curve is plotted, showing how the model's error decreases over time as it learns from the data. After training, the model is tested with a new example of flower features to predict its species. The prediction is made by feeding the input data into the trained model, which outputs a probability

distribution across the 3 classes. The species with the highest probability is selected as the predicted class. The plot of the loss curve reflects the model's learning process, with a steady decline indicating that the model is effectively reducing error as it trains. The final prediction provides the species of the Iris flower based on the given feature values, demonstrating the model's classification capability.

6 Hidden layers, Learning rate 1.0







Explanation:

The previous code has been adjusted to contain different hidden layer size and learning rate, to demonstrate the effect of the training dynamics and performance of the MLP model on the Iris dataset.

In this case, the MLP model is initialized with 6 neurons in the hidden layer (compared to the previous model, which had 12 neurons) and a learning rate of 1.0 (which is higher than the previous value of 0.05). The model is trained for 3000 epochs using the cross-entropy loss function to minimize the error between the predicted and actual species labels. The training loss is tracked over epochs, and a plot is generated to visualize the model's performance.

The graph of the loss curve for this setup is expected to behave differently due to the changes in the model's configuration. With a smaller hidden layer (6 neurons instead of 12), the model has fewer parameters, which may limit its capacity to capture complex patterns in the data, potentially resulting in slower convergence or higher final loss. A higher learning rate (1.0) increases the step size during weight updates, which causes the model to converge more quickly or overshoot the optimal solution, leading to fluctuations in the loss curve. This results in a less smooth decline in the loss where the model might not fully converge to a minimum value, as the higher learning rate prevents it from settling into the optimal solution. After training, the model makes a prediction for a new example using the trained weights. The predicted species is determined by the class with the highest output probability.

In summary, the smaller hidden layer and higher learning rate influence the model's ability to learn effectively. The loss curve shows a more erratic decrease, indicating that the model's performance is more sensitive to the training dynamics. The combination of these hyperparameters leads to a model that may not perform as well as a larger hidden layer or lower learning rate, as it struggles to fully capture the complexity of the dataset.

Enterprise Data found online: Hidden layers 16, learning rate 0.05 with cross entropy:

jupyter XOR_MLP v1 (1) Last Checkpoint: 56 minutes ago

File Edit View Run Kernel Settings Help

Code

JupyterLab Python 3 (ipykernel)

```
# Load and preprocess the dataset
def load_enterprise_data(file_path):
    df = pd.read_csv("annual-enterprise-survey-2023-financial-year-provisional-size-bands.csv")

    # Drop rows with missing values
    df = df.dropna()

    # Select relevant columns for training
    numerical_cols = ['value']
    categorical_cols = ['industry_code_ANZSIC', 'industry_name_ANZSIC', 'rme_size_grp']

    for col in numerical_cols:
        df[col] = pd.to_numeric(df[col], errors='coerce')  # Convert to numeric
    df = df.dropna(subset=numerical_cols)  # Drop rows where conversion failed

    # Normalize numerical data
    numerical_data = (df[numerical_cols] - df[numerical_cols].mean()) / df[numerical_cols].std()

    # One-hot encode categorical columns
    categorical_data = pd.get_dummies(df[categorical_cols], drop_first=True)

    # Combine the data
    X = pd.concat([numerical_data, categorical_data], axis=1).values

    # Ensure that the data is float type
    X = X.astype(np.float32)  # Convert the entire dataset to float32

    # One-hot encode the target variable (industry code)
    y = pd.get_dummies(df['industry_code_ANZSIC']).values
```

jupyter XOR_MLP v1 (1) Last Checkpoint: 57 minutes ago

File Edit View Run Kernel Settings Help

Code

JupyterLab Python 3 (ipyke

```
# Training function
def train_enterprise_mlp():
    print("Starting training...")

    # Initialize the MLP model
    mlp_enterprise = MLP(input_size=X.shape[1], hidden_layers=[16], output_size=y.shape[1])

    # Train the model
    losses = mlp_enterprise.train(X, y, epochs=3000, learning_rate=0.05, loss_function='cross_entropy')

    # Plot the loss curve after training
    import matplotlib.pyplot as plt
    plt.plot(losses)
    plt.title('Loss Curve')
    plt.xlabel('Epochs')
    plt.ylabel('Loss')
    plt.show()

# Make sure to call the training function
train_enterprise_mlp()
```

```
Starting training...
Training started for 3000 epochs.
X shape: (150, 4), y shape: (150, 3)
Epoch: 0
Epoch 0/3000, Loss: 2.1001
Epoch: 1
Epoch: 2
Epoch: 3
```

jupyter XOR_MLP v1 (1) Last Checkpoint: 53 minutes ago

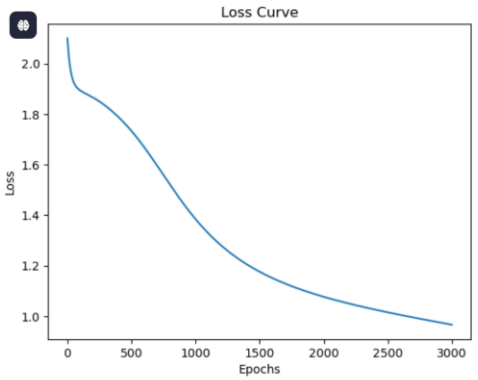
File Edit View Run Kernel Settings Help

Trusted

Code

JupyterLab Python 3 (ipykernel)

```
Epoch: 2999
```



Explanation:

This code trains a Multi-Layer Perceptron on the "Annual Enterprise Survey" dataset for 2023, which contains financial and industry-related data. The goal is to predict the industry code (industry_code_ANZSIC) based on numerical and categorical features, such as the enterprise size, industry name, and financial value (value). The dataset is first preprocessed by handling missing values, normalizing numerical columns (like value), and one-hot encoding categorical columns (industry_code_ANZSIC, industry_name_ANZSIC, rme_size_grp). After preprocessing, the features are combined into a matrix X (input features), and the target labels are encoded into one-hot format in the array y.

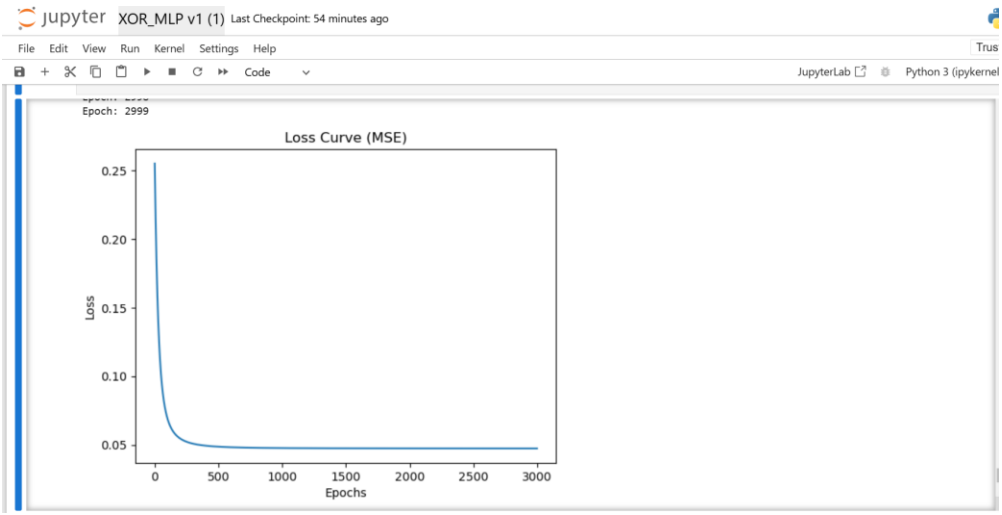
The MLP model is defined with an input layer size matching the number of features in X, one hidden layer with 16 neurons, and an output layer with a number of neurons equal to the number of unique industry codes. The model is trained using the cross-entropy loss function for 3000 epochs with a learning rate of 0.05.

During training, the forward pass calculates activations at each layer using the sigmoid activation function. In the backward pass, the gradient of the loss is calculated with respect to the weights and biases, and the weights are updated using gradient descent. The loss is tracked at each epoch, and the loss curve is plotted to visualize the model's training progress.

The loss curve provides insights into the model's performance over time. The loss decreases steadily as the model learns from the data.

In summary, the code trains an MLP on a preprocessed dataset, and the output loss curve reflects how the model's predictions improve over time. The steady decrease in the loss curve highlights that the model is learning effectively.

Enterprise Data: Hidden layers 16, learning rate 0.05 with mean squared error



Explanation:

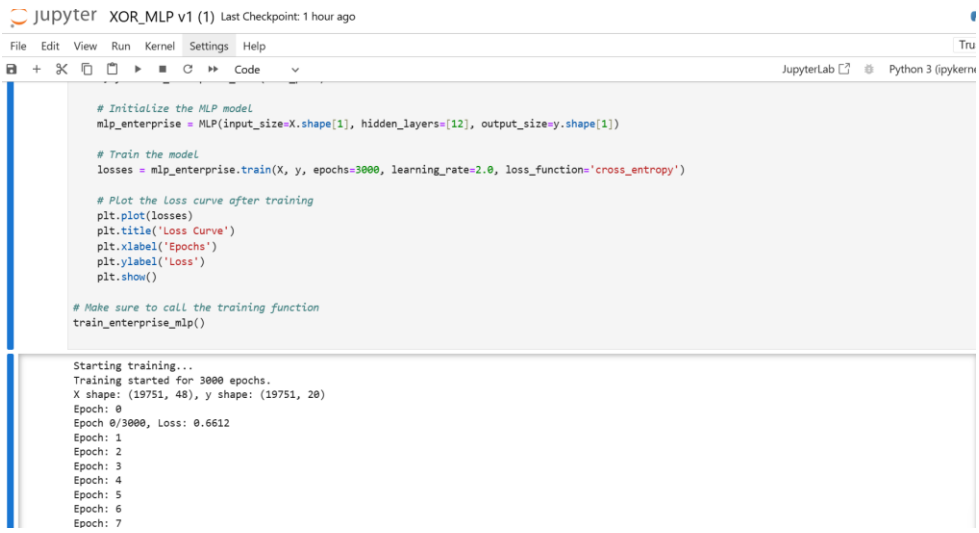
Similar to the last, this code trains the MLP model using the "Annual Enterprise Survey" dataset from 2023 to predict the industry_code_ANZSIC based on features such as enterprise size (rme_size_grp), industry name (industry_name_ANZSIC), and financial value (value).

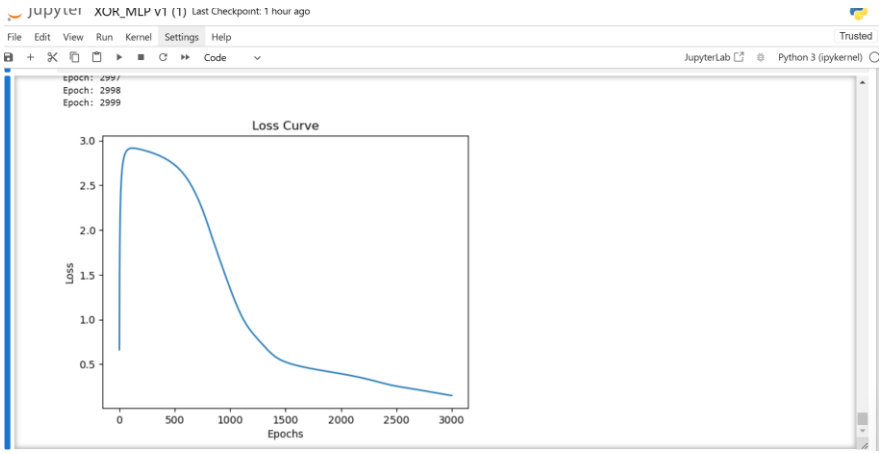
The MLP model is defined with an input layer size corresponding to the number of features in X, a single hidden layer with 16 neurons, and an output layer that matches the number of unique industry codes. The model is trained for 3000 epochs with a learning rate of 0.05 using the Mean Squared Error (MSE) loss function, as opposed to the cross-entropy loss used in the previous example.

The use of MSE as a loss function indicates that the model is being trained with a regression approach, which measures the average squared differences between predicted and actual values. Cross-entropy loss, on the other hand, is typically used for classification tasks and measures how well the predicted probabilities match the true class distributions. The choice of MSE is less suitable for classification tasks because it does not capture the categorical nature of the target variable (industry_code_ANZSIC). This can lead to slower convergence or less efficient training as the model may struggle to minimize the loss effectively.

As a result, the loss curve shows slower reductions in loss compared to using cross-entropy for classification problems. In summary, while MSE can be applied in some cases for classification, it is not the ideal choice, and this mismatch may result in a less optimal loss curve during training.

Enterprise Data: Hidden layers 12, learning rate 2.0 with cross entropy





Explanation:

The MLP model in this code consists of an input layer with a size corresponding to the number of features in X, a hidden layer with 12 neurons, and an output layer that matches the number of unique industry_code_ANZSIC values. The model is trained for 3000 epochs with a learning rate of 2.0, using the cross-entropy loss function. This loss function is commonly used in classification problems as it measures how well the predicted probabilities match the actual class labels. In this case, it's appropriate for predicting categorical outcomes like industry codes.

In comparison to the previous example with 16 neurons and an 0,05 learning rate, the same model can be trained with different hyperparameters. The choice of a larger hidden layer (16 neurons vs. 12 neurons) increases the model's capacity to learn more complex patterns in the data, as it provides more neurons for each layer. A smaller learning rate of 0.05 slows down the learning process, allowing the model to make smaller, more gradual updates to the weights.

This slower learning rate prevents overshooting the optimal solution, leading to a more stable and possibly more precise convergence, but it also increases the time required to train the model.

The forward pass computes activations using the sigmoid function, which maps values between 0 and 1, while the backward pass computes gradients of the loss with respect to the weights and biases and updates them using backpropagation. In the 12-neuron, 2.0 learning rate model, the updates may be faster, potentially causing the model to converge more quickly, but possibly with less precision. In contrast, the 16-neuron, 0.05 learning rate model, while slower, might result in better long-term learning, especially for more complex datasets, as it allows for more fine-grained weight adjustments.

The training function train_enterprise_mlp() handles the loading of the dataset, initialization of the MLP model, training, and plotting the loss curve. The loss curve shows the model's performance over time. With the smaller learning rate (0.05) and more neurons (16), the plot shows a smoother and slower reduction in loss, reflecting a more gradual improvement in model performance. Conversely, with the larger learning rate (2.0) and fewer neurons (12), the plot shows faster decreases in loss, but with more fluctuations, as the model struggles with overshooting or unstable convergence.

In summary, the two configurations (12 neurons with a 2.0 learning rate versus 16 neurons with a 0.05 learning rate) offer different trade-offs. The model with 16 neurons and a smaller learning rate is likely to be more stable and precise, but slower to converge, while the model with 12 neurons and a larger learning rate converges more quickly but with less stability, leading to a suboptimal solution.